The LC-3 Instruction Set Architecture

CSE 100/1000/1100: Introduction to Computer Engineering

Instructor:
Fatma CORUT ERGİN

Slides adapted from

- *McGraw Hill Companies, and*
- *Walid A. Najjar & Brian J. Linard*

slides

Instruction Set Architecture

ISA = All of the *programmer-visible* components and operations of the computer

- **memory organization**
  - **address space -- how may locations can be addressed?**
  - **addressability -- how many bits per location?**
- **register set**
  - **how many? what size? how are they used?**
- **instruction set**
  - **opcodes**
  - **data types**
  - **addressing modes**

ISA provides all information needed for someone that wants to write a program in machine language (or translate from a high-level language to machine language).

LC-3 ISA Overview

Memory

- address space: 2^{16} locations (16-bit addresses)
- addressability: 16 bits (word-addressable)

total memory size = $2^{16} * 16 = 2^{20}$ bits = 2^{17} bytes = 2^{7} kbytes = 128 KB

Registers

- 8 general-purpose registers: R0 - R7
  - each 16-bits wide
- other registers
  - not directly addressable, but used by (and affected by) instructions
  - PC (program counter),
  - 1-bit Condition Code Registers: N, Z, P

Instructions

- 16-bit instructions, with 4-bit opcodes

LC-3 Overview: Instruction Set

Opcodes

- **15 opcodes**
- ***Operate* instructions: ADD, AND, NOT**
- ***Data movement* instructions: LD, LDI, LDR, LEA, ST, STR, STI**
- ***Control* instructions: BR, JSR/JSRR, JMP, RTI, TRAP**
- **some opcodes set/clear *condition codes*, based on result:**
  - **N = negative, Z = zero, P = positive**

Data Types

- **16-bit 2's complement integer**

Addressing Modes

- **How is the location of an operand specified?**
- **non-memory addresses: *immediate*, *register***
- **memory addresses: *PC-relative*, *indirect*, *base+offset***

LC-3 Instructions

Operate

- **Manipulate data directly**
  - **ADD, AND, NOT**

Data Movement

- **Move data between memory and registers**
  - **LD, LDI, LDR, LEA, ST, STI, STR**

Control

- **Change the sequence of instruction execution**
  - **BR, JMP/RET, JSR/JSSR, TRAP, RTI**

LC 3 Instructions

LC-3 Instruction word: 16 bits

- **Opcode**
  - **IR[15:12]: 4 bits allow 16 instructions**
  - **specifies the instruction to be executed**
- **Operands**
  - **IR[11:0]: contains specifications for:**
  - **Registers: 8 GPRs (i.e. require 3 bits for addressing)**
  - **Address Generation bits: Offset (11 or 9 or 6 bits) (more later)**
  - **Immediate value: 5 bits**

Examples

- **ADD DR, SR1, SR2 ; DR ← (SR1) + (SR2)**

  [15:12] [11:9] [8:6] [2:0]

- **LDR DR, BaseR, Offset ; DR ← Mem[BaseR + Offset]**

  [15:12] [11:9] [8:6] [5:0]

Addressing Modes

Note: the effective address (ea) is the memory location of the operand

The LC-3 supports five addressing modes:

- **the operand is located:**
  - **in the instruction itself (immediate)**
  - **in a register**
  - **in memory:**
    - **the ea is encoded in the instruction (direct, or PC-relative)**
    - **a pointer to the ea is encoded in the instruction (indirect)**
    - **a pointer to the ea is stored in a register (relative, or base+offset)**

Operate Instructions

Arithmetic and Logic

- Arithmetic: add, subtract, multiply, divide (the LC-3 only has ADD)
- Logic: and, or, not, xor (the LC-3 only has AND, NOT)

LC-3: ADD, AND, NOT

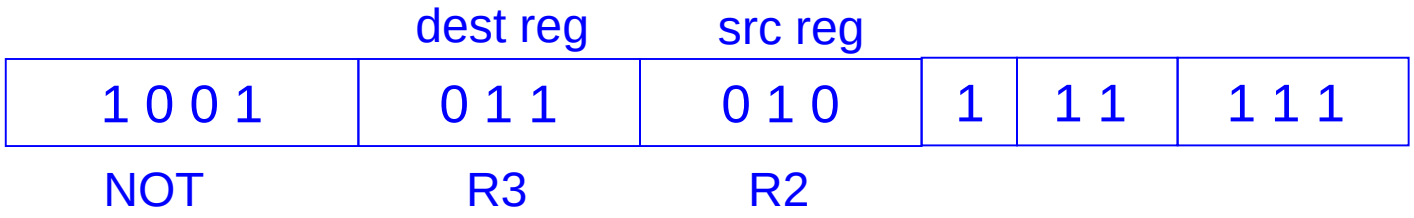
Source and destination operands are registers

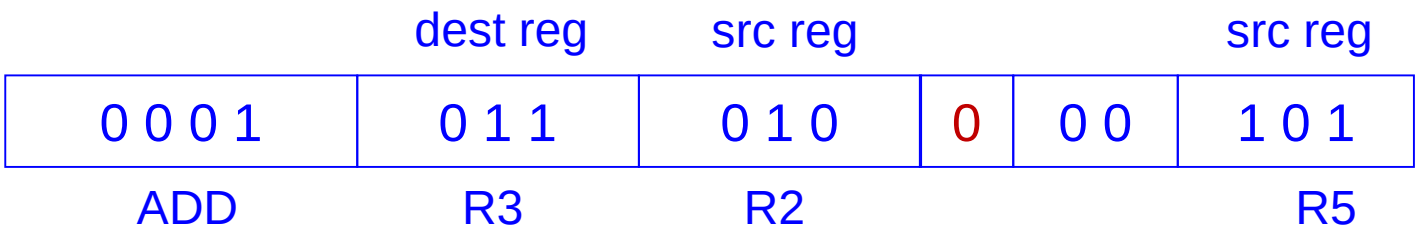
- These instructions ***do not*** reference memory.
- ADD and AND can use "immediate" mode,
  where one operand is hard-wired into the instruction.

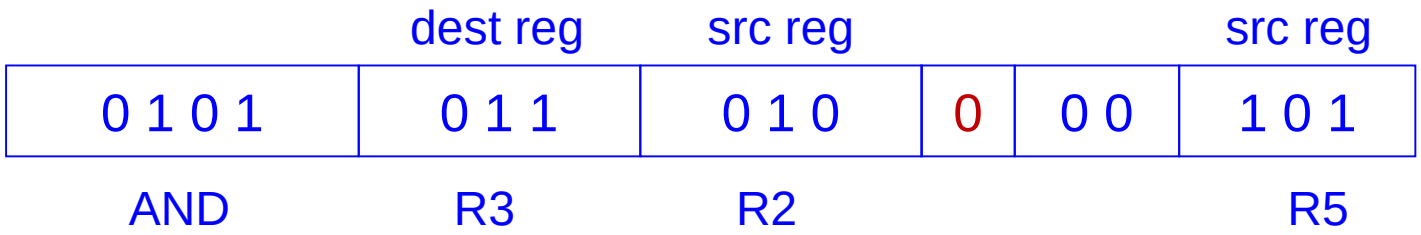
Will show dataflow diagram with each instruction.

- illustrates ***when*** and ***where*** data moves
  to accomplish the desired operation

Operate Instructions







Operate Instructions

NOT (unary operator)

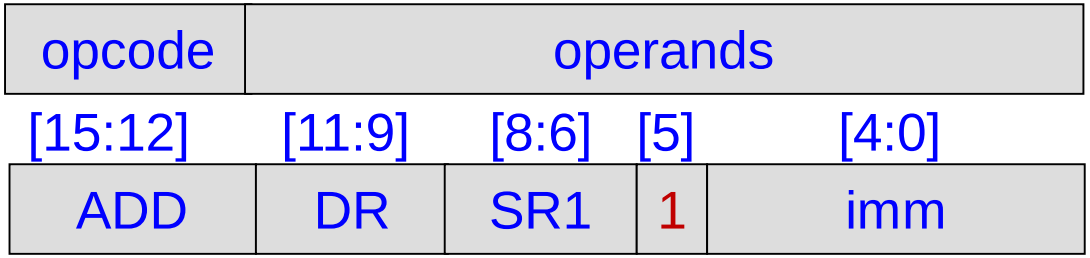
- destination register in IR[11:9] and a single source register in IR[8:6].
- bits IR[5:0] are all 1s.

ADD & AND (binary operators)

- destination register in IR[11:9], one source register in IR[8:6]
- other source operand may be:
  - ➢ immediate addressing mode:
    - – if bit IR[5] = 1, bits IR[4:0] specify the other source number directly, as a 5 bit 2's complement integer, which is sign extended (SEXT) to 16 bits.
  - ➢ register addressing mode:
    - – if bit IR[5] = 0, bits IR[2:0] specify a register for the second source
    - – bits IR[4:3] = 00

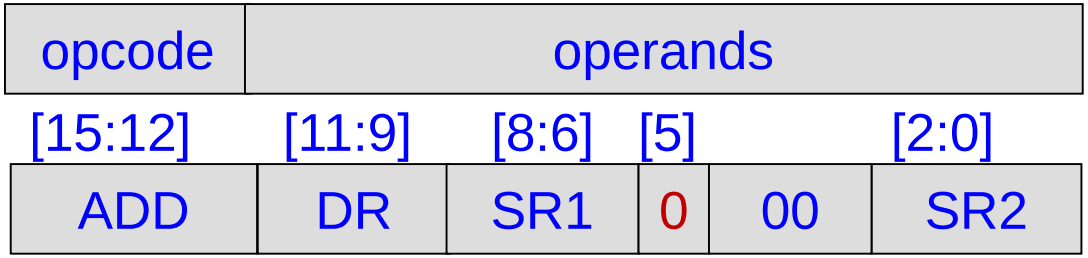
Immediate & Register Operands

Immediate



- **If bit 5 = 1, the value in IR[4:0] ("immediate") is *sign extended (SEXT)* to 16 bits and added to the contents of the source register SR1 (IR[8:6]).**

Register



- **if bit 5 = 0, the contents of source register SR2 (IR[2:0]) are added to the contents of source register SR1 (IR[8:6]).**

In both cases, the result goes to the destination register DR (IR[11:9]).

NOT: Bitwise Logical NOT

Assembler Instruction

```
NOT  DR, SR
```

Encoding

```
1001  DR  SR  111111    ; DR = NOT(SR)
```

Example

```
NOT   R2, R6
```

- **Note: Condition codes are set.**

NOT Instruction Cycle and Data Path

NOT R3, R5 → 1001 011 101 111111

FETCH:

MAR = PC, PC = PC + 1
MDR = Mem[MAR]
IR = MDR

DECODE:

IR[15:12] = 1001 (opcode)

EVALUATE ADDRESS:

not used

FETCH OPERANDS:

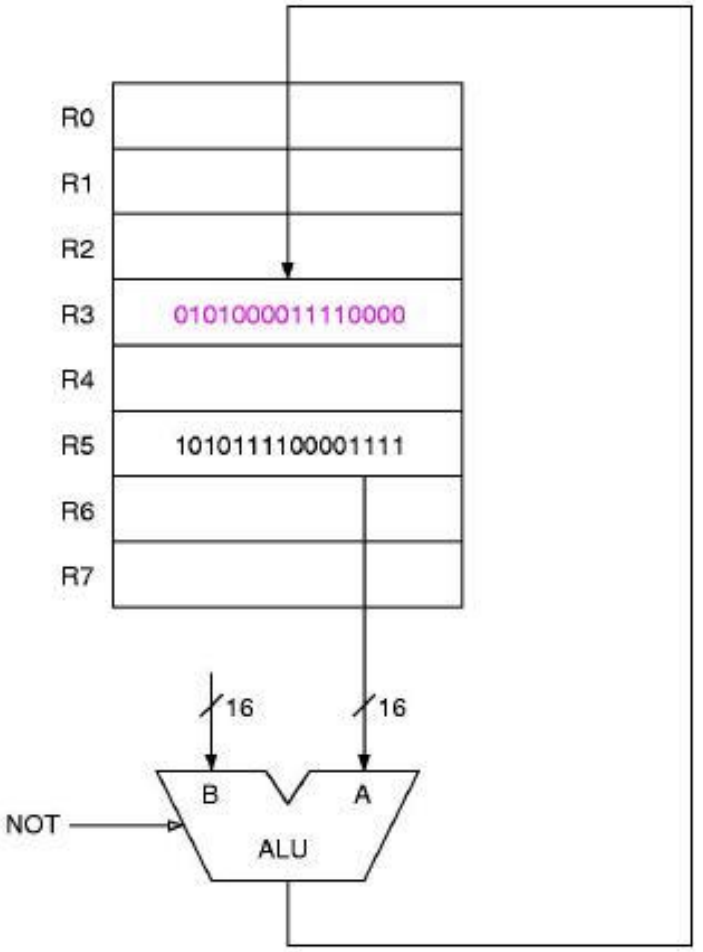
The value of R5 is fetched from register file
→ operand is 1010111100001111

EXECUTE:

NOT(1010111100001111) operation is performed in the ALU

STORE RESULT:

Result is written to destination register, R3



ADD: Two's complement 16-bit Addition

Assembler Instruction

```
ADD  DR, SR1, SR2
ADD  DR, SR1, imm5
```

Encoding

```
0001  DR  SR1  0  00  SR2   ; DR = SR1 + SR2
0001  DR  SR1  1   imm5     ; DR = SR1 + SEXT(imm5)
```

Examples

```
ADD   R1, R4, R5
ADD   R1, R4, # -2
```

- **Note: Condition codes are set**

AND: Bitwise Logical AND

Assembler Instruction

```
AND  DR, SR1, SR2
AND  DR, SR1, imm5
```

Encoding

```
0101  DR  SR1  0  00  SR2    ; DR = SR1  AND SR2
0101  DR  SR1  1   imm5      ; DR = SR1 AND SEXT(imm5)
```

Examples

```
AND   R2, R3, R6
AND   R2, R2, #0    ; Clear R2 to 0
```

Question: ***if the immediate value is only 6 bits, how can it mask the whole of R2?***

- **Note: Condition codes are set.**

ADD/AND Instruction Cycle and Data Path

ADD R1, R4, # -2 → 0001 001 100 1 11110

FETCH:

- MAR = PC, PC = PC + 1
- MDR = Mem[MAR]
- IR = MDR

DECODE:

- IR[15:12] = 0001 (opcode)
- IR[5] = 1

  →operands are a register and an immediate value

EVALUATE ADDRESS:

not used

FETCH OPERANDS:

- Immediate value (IR[4:0]) is sign extended
- The value of R4 is fetched from the register file

→ operands are

→ 0000000000000110 (x0006)

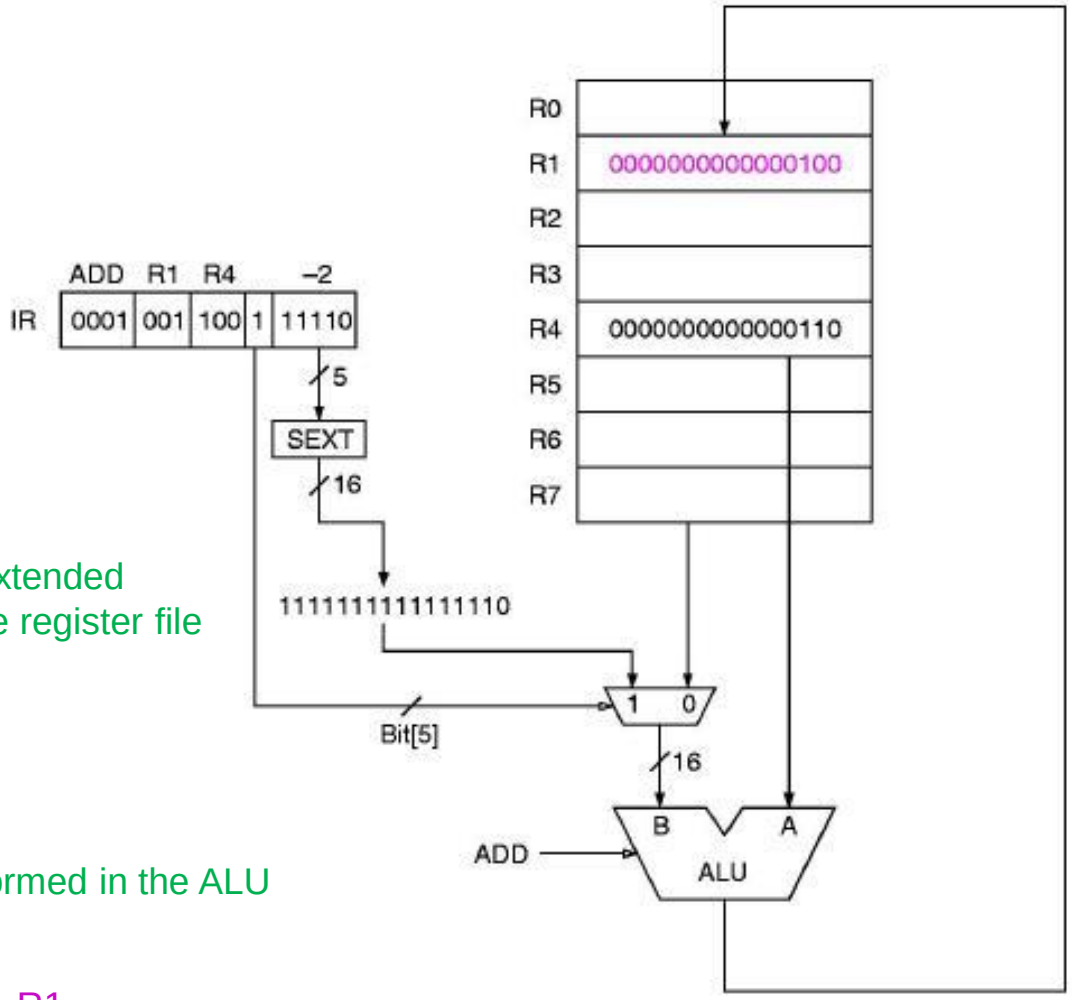
→ 1111111111111110 (xFFFE)

EXECUTE:

ADD(x0006, xFFFE) operation is performed in the ALU

STORE RESULT:

Result is written to destination register, R1



ADD/AND Instruction Cycle and Data Path

AND R1, R4, R5 → 0101 001 100 0 00 011

FETCH:

- MAR = PC, PC = PC + 1
- MDR = Mem[MAR]
- IR = MDR

DECODE:

- IR[15:12] = 0101 (opcode)
- IR[5] = 0
  →operands are two registers

EVALUATE ADDRESS:

not used

FETCH OPERANDS:

- The values of R4 and R3 are fetched from the register file

→ operands are

→ 0000000000000110 (x0006)

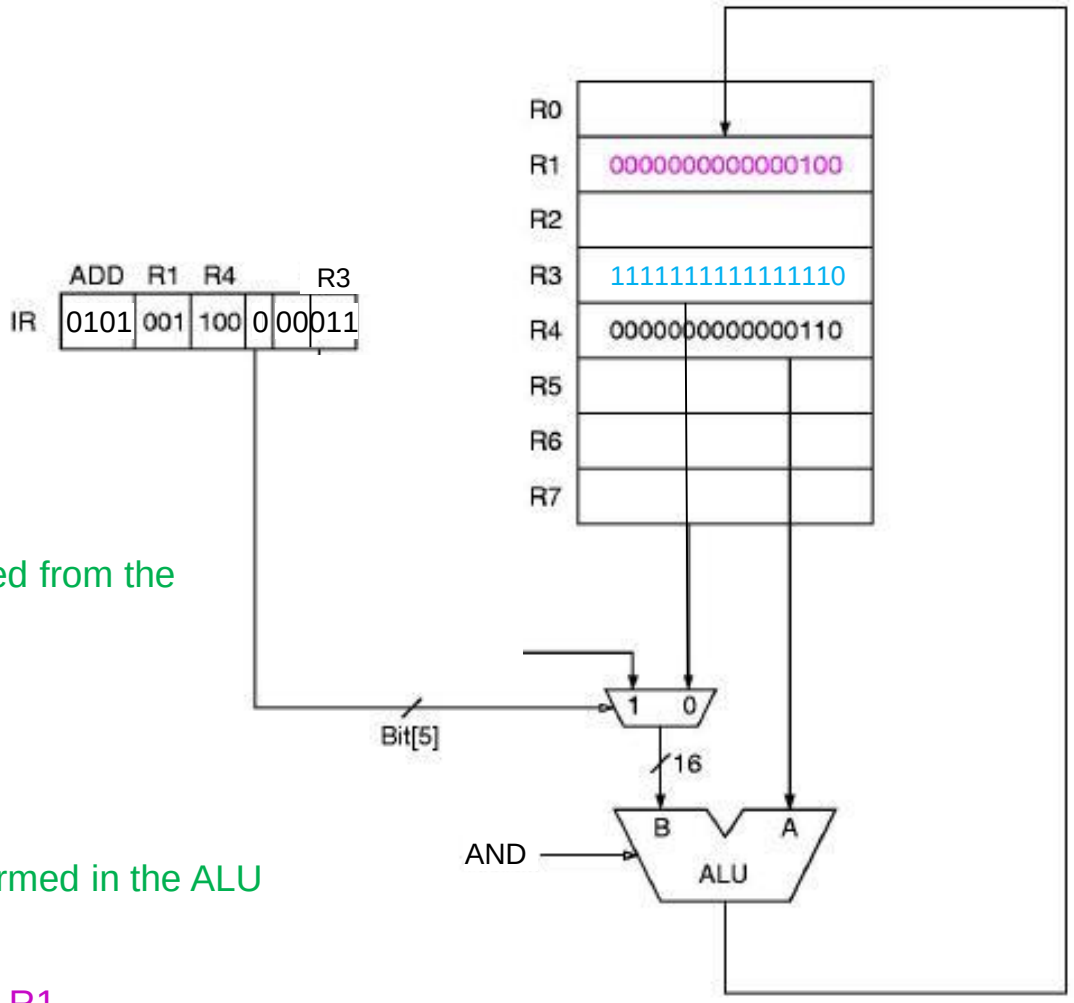
→ 1111111111111110 (xFFFE)

EXECUTE:

AND(x0006, xFFFE) operation is performed in the ALU

STORE RESULT:

Result is written to destination register, R1



Using Operate Instructions

With only ADD, AND, NOT…

- **How do we subtract?**
- **How do we OR?**
- **How do we copy from one register to another?**
- **How do we initialize a register to zero?**

Data Movement Instructions

Move Data

- **from register to memory => store**
  - **selected register is Source**
- **from memory to register => load**
  - **selected register is Destination**
- **Cannot move data from memory to memory**

LC-3 Load/Store Instructions

- **LD, LDI, LDR, LEA, ST, STI, STR**
- **Format:**

opcode	DR or SR	Address generator bits

LC-3 Load/Store Addressing Modes

- **immediate:** **LEA** **(no memory access is required)**

  DR = (PC) + SEXT(IR[8:0])

- **direct or PC-Relative: LD & ST**

  MemAddr = (PC) + SEXT(IR[8:0])

- **indirect: LDI & STI**

  MemAddr = Mem[(PC) + SEXT(IR[8:0])]

- **base+offset (Relative): LDR & STR** **(BaseReg is specified by IR[8:6])**

  MemAddr = BaseReg + SEXT(IR[5:0])

Remember that PC is incremented in the FETCH phase

LEA: Load Effective Address

Assembler Instruction

LEA DR, LABEL

Encoding

1110 DR PCoffset9 ; DR = PC + SEXT(PCoffset9)

Example

LEA R5, # -3

Assume memory address x4018 contains the instruction ***LEA R5, #-3***

- PC = x4019 after the FETCH phase

then, after the instruction ***LEA R5, #-3*** is executed,

R5 will contain x4016 (R5 = x4019 + xFFFD = x4016)

Note 1: No memory access is required!

Note 2: Condition codes are set.

LEA Instruction Cycle and Data Path

LEA R5, # -3 → 1110 101 11111101

FETCH:

- MAR = PC, PC = PC + 1
- MDR = Mem[MAR]
- IR = MDR

DECODE:

IR[15:12] = 1110 (opcode)

→ The operands are PC and an immediate value

EVALUATE ADDRESS:

not used

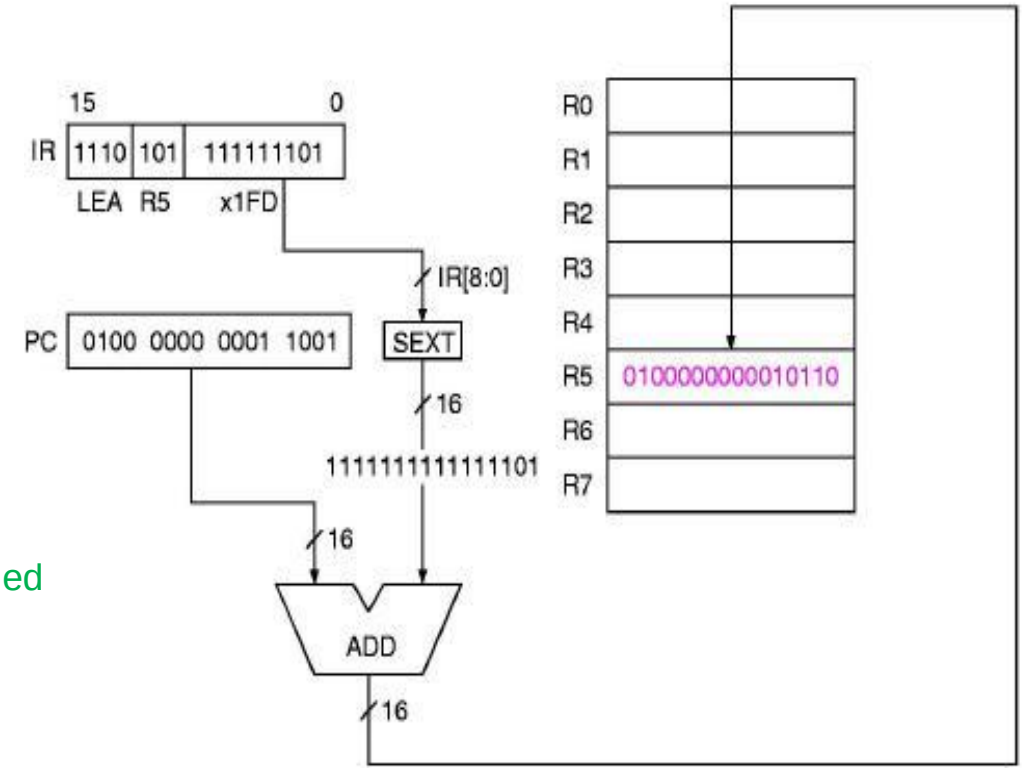
FETCH OPERANDS:

Immediate value (IR[8:0]) is sign extended

→ operands are

→ 1111111111111101 (xFFFD)

→ 0100000000011001 (x4019)



EXECUTE:

ADD(x4019, xFFFD) operation is performed in the ALU

STORE RESULT:

Result is written to destination register, R5

LC-3 Load/Store Addressing Modes

- **immediate: LEA** (no memory access is required)

  DR = (PC) + SEXT(IR[8:0])

- **direct (PC-Relative): LD & ST**

  MemAddr = (PC) + SEXT(IR[8:0])

- **indirect: LDI & STI**

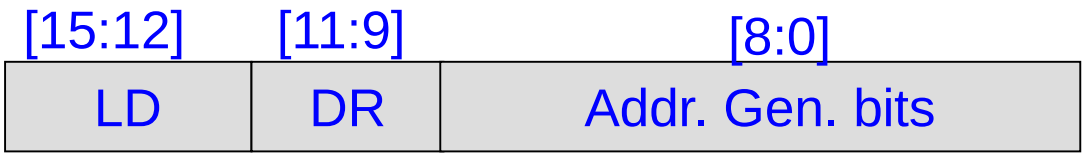
  MemAddr = Mem[(PC) + SEXT(IR[8:0])]

- **base+offset (Relative): LDR & STR** (BaseReg is specified by IR[8:6])

  MemAddr = BaseReg + SEXT(IR[5:0])

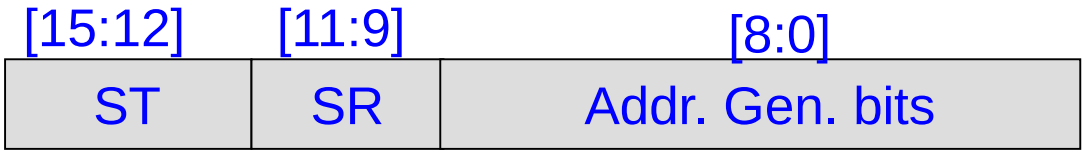
Remember that PC is incremented in the FETCH phase

Direct addressing (PC-Relative)



memory address = (PC) + SEXT(IR[8:0])

DR = Mem[memory address]



memory address = (PC) + SEXT(IR[8:0])

Mem[memory address] = SR

LD: Load Direct

Assembler Instruction

LD DR, LABEL

Encoding

0010 DR PCoffset9 ; DR = Mem[PC + SEXT(PCoffset9)]

Example

LD R2, x1AF

Assume:

- memory address x4018 contains the instruction ***LD R2, x1AF***
  - PC = x4019 after the FETCH phase
- memory address x3FC8 contains the value 5

then, after the instruction ***LD R2, x1AF*** is executed,

R2 will contain value 5 (R2 = Mem[x4019+xFFAF] = Mem[x3FC8] = 5)

Note: Condition codes are set.

LD Instruction Cycle and Data Path

LD R2, x1AF → 0010 010 110101111

FETCH:

- MAR = PC, PC = PC + 1
- MDR = Mem[MAR]
- IR = MDR

DECODE:

IR[15:12] = 0010 (opcode)

EVALUATE ADDRESS:

IR[8:0] is sign extended

MAR = PC + Sext(IR[8:0]) (step 1)

FETCH OPERANDS:

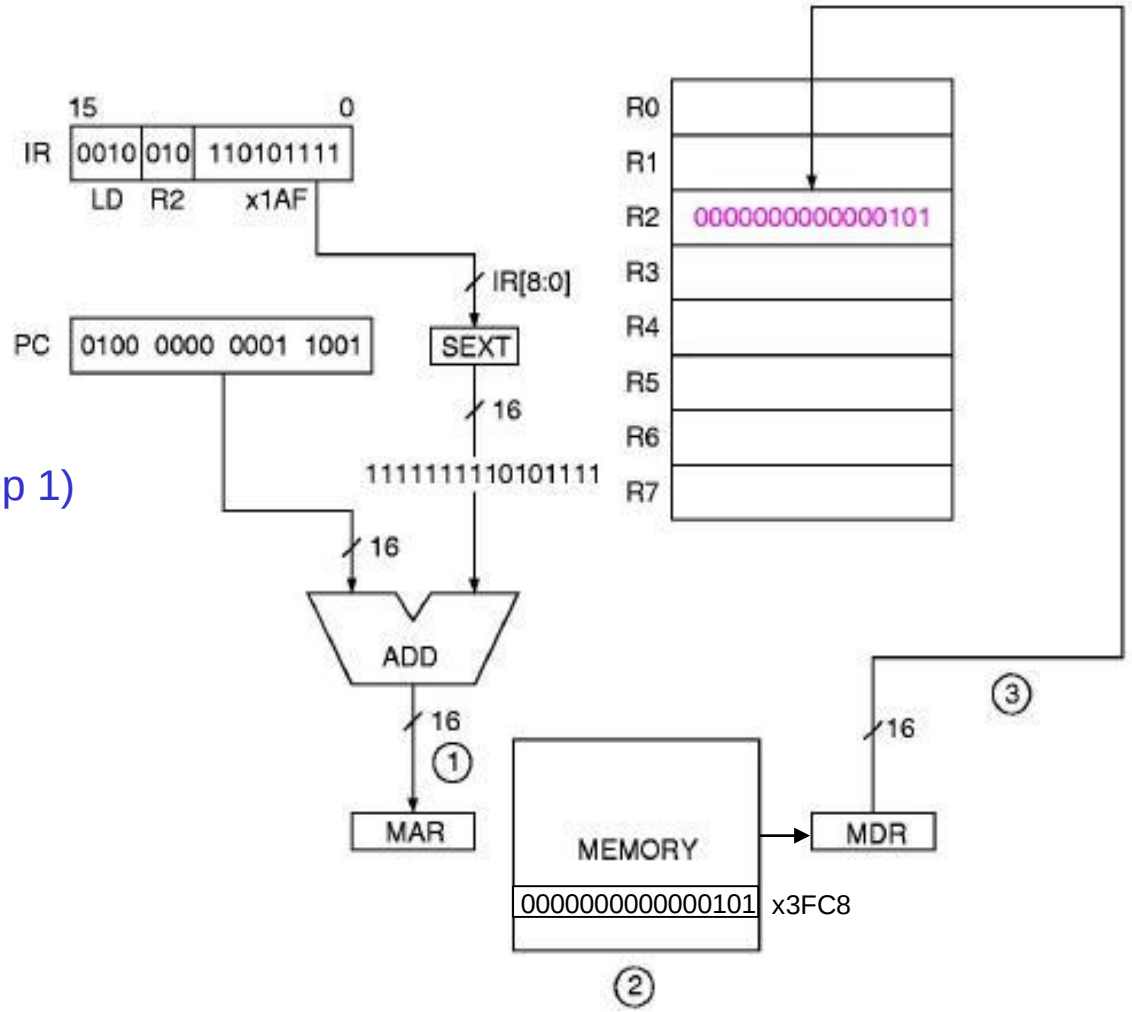
MDR = Mem[MAR] (step 2)

EXECUTE:

not used

STORE RESULT:

R2 = MDR (step 3)



ST: Store Direct

Assembler Instruction

ST SR, LABEL

Encoding

0011 SR PCoffset9 ; Mem[PC+SEXT(PCoffset9)] = SR

Example

ST R2, x1AF

Assume:

- memory address x4018 contains the instruction ***`ST R2, x1AF`***
  - PC = x4019 after the FETCH phase
- Register R2 contains the value 5

then, after the instruction ***`ST R2, x1AF`*** is executed,

memory address x3FC8 will contain the value 5 (Mem[x3FC8] = R2 = 5)

Notes: Condition codes are NOT set.

LC-3 Load/Store Addressing Modes

- **immediate: LEA** (no memory access is required)

  DR = (PC) + SEXT(IR[8:0])

- **direct (PC-Relative): LD & ST**

  MemAddr = (PC) + SEXT(IR[8:0])

- **indirect: LDI & STI**

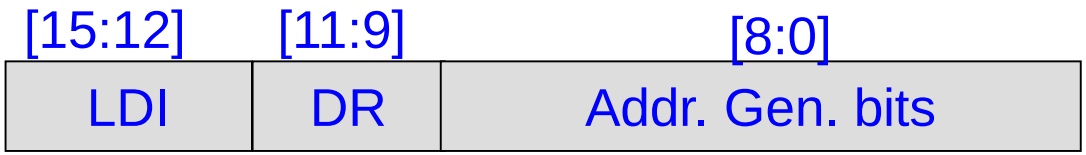
  MemAddr = Mem[(PC) + SEXT(IR[8:0])]

- **base+offset (Relative): LDR & STR** (BaseReg is specified by IR[8:6])

  MemAddr = BaseReg + SEXT(IR[5:0])

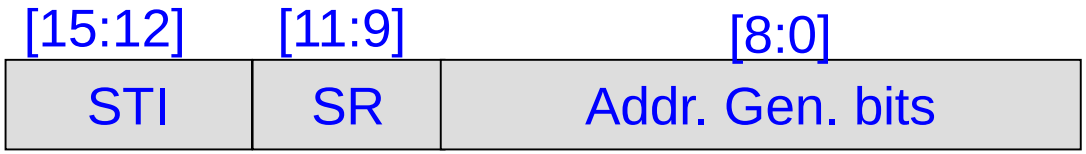
Remember that PC is incremented in the FETCH phase

Indirect Addressing



memory address = (PC) + SEXT(IR[8:0])

DR = Mem[Mem[memory address]]



memory address = (PC) + SEXT(IR[8:0])

Mem[Mem[memory address]] = SR

- **Note that the memory has to be accessed twice to get the actual operand.**

LDI: Load Indirect

Assembler Instruction

LDI DR, LABEL

Encoding

1010 DR PCoffset9 ; DR = Mem[Mem[PC + SEXT(PCoffset9)]]

Example

LDI R3, x1CC

Assume:

- memory address x4A1B contains the instruction ***`LDI R3,x1CC`***
  - PC = x4A1C after the FETCH phase
- memory address x49E8 contains the value x2110
- memory address x2110 contains the value -1

then, after the instruction ***`LDI R3,x1CC`*** is executed,

R3 will contain value -1

Note: Condition codes are set.

LDI Instruction Cycle and Data Path

LDI R3, x1CC → 1010 011 111001100

FETCH:

- MAR = PC, PC = PC + 1
- MDR = Mem[MAR]
- IR = MDR

DECODE:

IR[15:12] = 1010 (opcode)

EVALUATE ADDRESS:

IR[8:0] is sign extended

MAR = PC + Sext(IR[8:0]) = x49E8(step 1)

MDR = Mem[MAR] = x2110 (step2)

MAR = MDR = x2110 (step3)

FETCH OPERANDS:

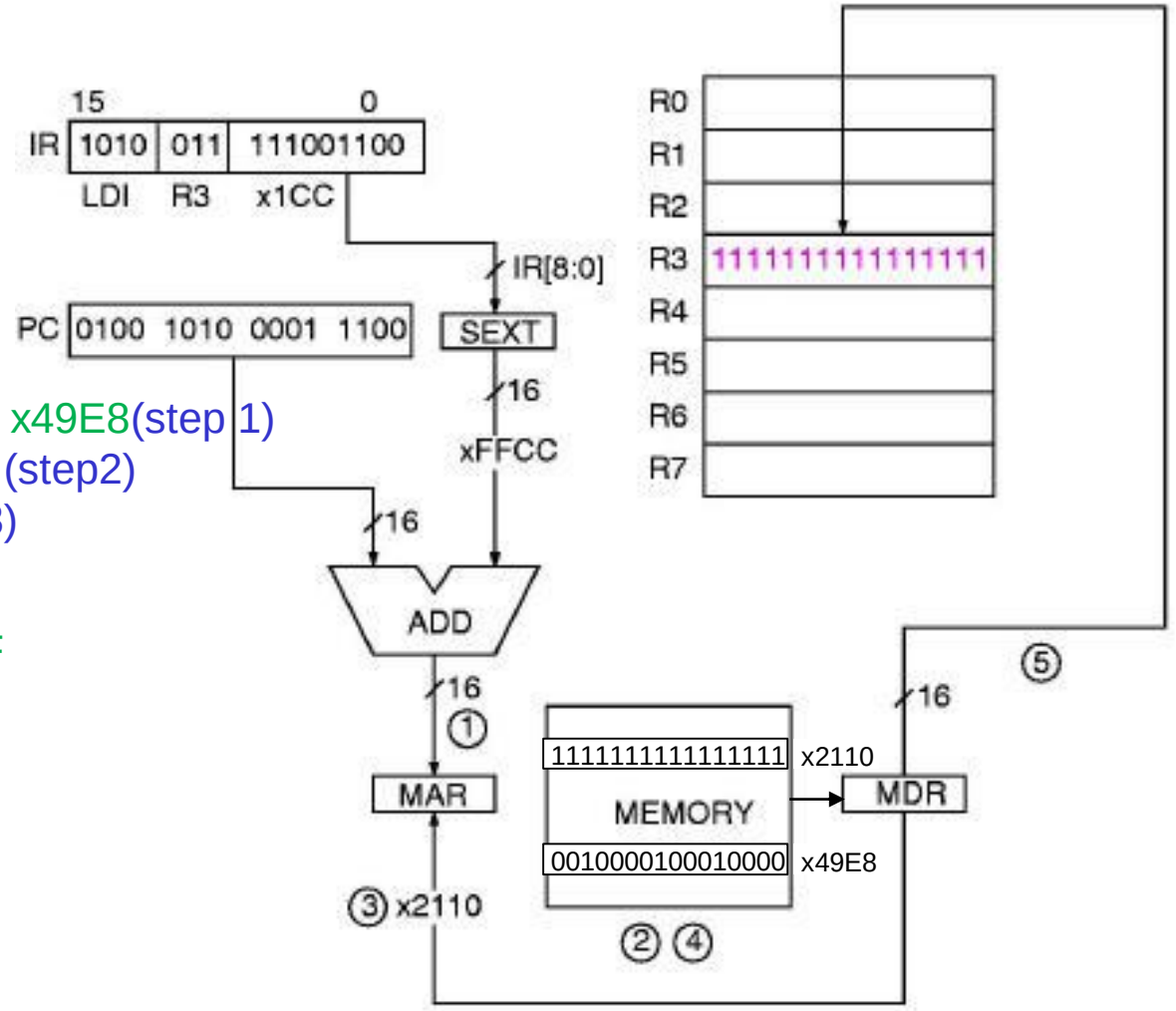
MDR = Mem[MAR] = xFFFF

EXECUTE:

not used

STORE RESULT:

R3 = MDR



STI: Store Indirect

Assembler Instruction

STI SR, LABEL

Encoding

1011 SR PCoffset9 ; Mem[Mem[PC + SEXT(PCoffset9)]] = SR

Example

STI R2, #2

Note: Condition codes are NOT set.

LC-3 Load/Store Addressing Modes

- **immediate: LEA** (no memory access is required)

  DR = (PC) + SEXT(IR[8:0])

- **direct (PC-Relative): LD & ST**

  MemAddr = (PC) + SEXT(IR[8:0])

- **indirect: LDI & STI**

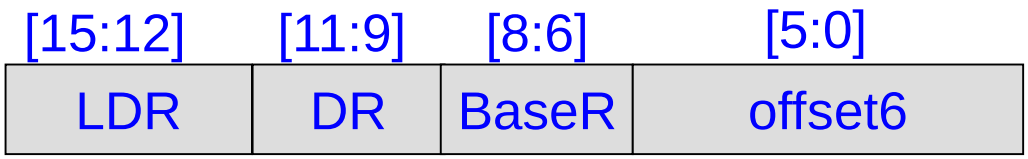
  MemAddr = Mem[(PC) + SEXT(IR[8:0])]

- **base+offset (Relative): LDR & STR** (BaseReg is specified by IR[8:6])

  MemAddr = BaseReg + SEXT(IR[5:0])

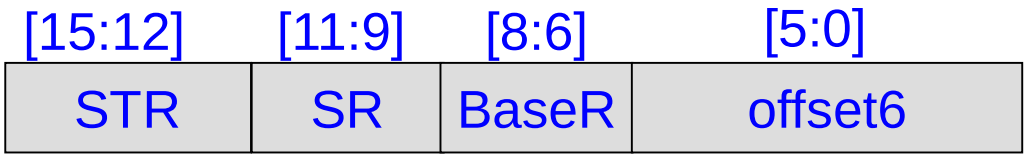
Remember that PC is incremented in the FETCH phase

Base+Offset (Relative) addressing



memory address = BaseR+ SEXT(IR[5:0])

DR = Mem[memory address]



memory address = BaseR+ SEXT(IR[5:0])

Mem[memory address] = SR

LDR: Load Base+Offset

Assembler Instruction

LDR DR, BaseR, offset6

Encoding

0110 DR BaseR offset6 ; DR = Mem[BaseR+SEXT(offset6)]

Example

LDR R1, R2, x1D

Assume:

- register R2 contains the value x2345
- memory address x2362 contains the value x0F0F

then, after the instruction ***`LDR R1, R2, x1D`*** is executed,

R1 will contain value x0F0F

Note 1: The 6 bit offset is a 2's complement number.
Note 2: Condition codes are set.

LDR Instruction Cycle and Data Path

LDR R1, R2, x1D → 1010 011 011 011101

FETCH:

- MAR = PC, PC = PC + 1
- MDR = Mem[MAR]
- IR = MDR

DECODE:

IR[15:12] = 0110 (opcode)

EVALUATE ADDRESS:

IR[5:0] is sign extended

MAR = R2 + Sext(IR[5:0]) = x2362 (step 1)

FETCH OPERANDS:

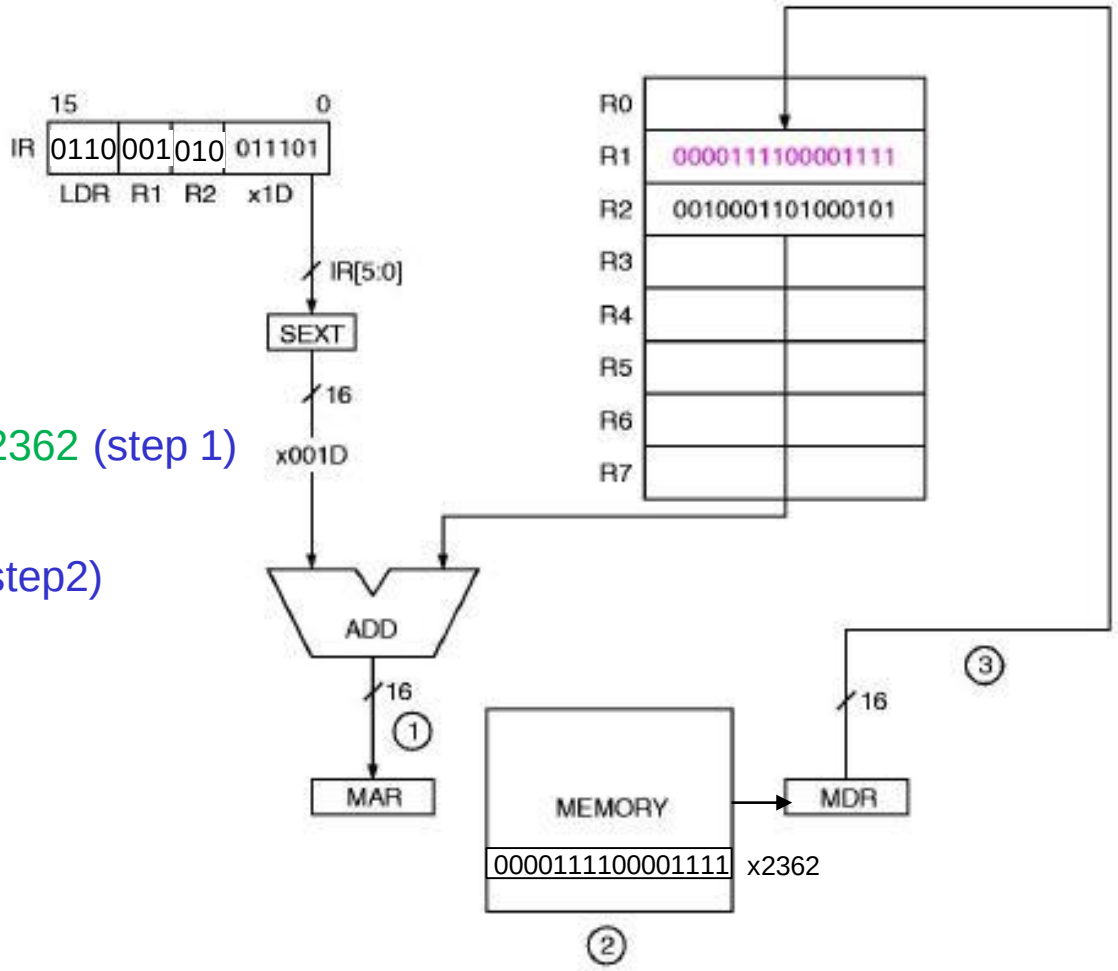
MDR = Mem[MAR] = x0F0F (step2)

EXECUTE:

not used

STORE RESULT:

R1 = MDR (step3)



STR: Store Base+Index

Assembler Instruction

STR SR, BaseR, offset6

Encoding

0111 SR BaseR offset6   ; Mem[BaseR+SEXT(offset6)] = SR

Example

```
STR   R2, R4, #15
```

Note: Condition codes are not set.

Example

Address	*Instruction*	*Assembler Instr.*
x30F6	1 1 1 0 0 0 1 1 1 1 1 1 1 0 1	*LEA R1, # -3*
x30F7	0 0 0 1 0 1 0 0 0 1 1 0 1 1 1 0	*ADD R2, R1, #14*
x30F8	0 0 1 1 0 1 0 1 1 1 1 1 1 0 1 1	*ST R2, x1FB*
x30F9	0 1 0 1 0 1 0 0 1 0 1 0 0 0 0 0	*AND R2, R2, #0*
x30FA	0 0 0 1 0 1 0 0 1 0 1 0 0 1 0 1	*ADD R2, R2, #5*
x30FB	0 1 1 1 0 1 0 0 0 1 0 0 1 1 1 0	*STR R2, R1, #14*
x30FC	1 0 1 0 0 1 1 1 1 1 1 1 0 1 1 1	*LDI R3, x1F7*

opcode

Example

Address	*Instruction*	*Comments*
x30F6	1 1 1 0 0 0 1 1 1 1 1 1 1 0 1	*R1 ← PC – 3 = x30F4*
x30F7	0 0 0 1 0 1 0 0 0 1 1 0 1 1 1 0	*R2 ← R1 + 14 = x3102*
x30F8	0 0 1 1 0 1 0 1 1 1 1 1 1 0 1 1	*M[PC - 5] ← R2* *M[x30F4] ← x3102*
x30F9	0 1 0 1 0 1 0 0 1 0 1 0 0 0 0 0	*R2 ← 0*
x30FA	0 0 0 1 0 1 0 0 1 0 1 0 0 1 0 1	*R2 ← R2 + 5 = 5*
x30FB	0 1 1 1 0 1 0 0 0 1 0 0 1 1 1 0	*M[R1+14] ← R2* *M[x3102] ← 5*
x30FC	1 0 1 0 0 1 1 1 1 1 1 1 1 0 1 1 1	*R3 ← M[M[x30F4]]* *R3 ← M[x3102]* *R3 ← 5*

opcode

Control Instructions

Used to change the sequence of instructions (by changing the Program Counter (PC))

Conditional Branch

- **if a specified condition is true, branch is *taken***
  - **signed offset is added to PC to yield new PC**
- **else, the branch is *not taken***
  - **PC is not changed, points to the next sequential instruction**

Unconditional Branch (or Jump)

- **always changes the PC**

Condition Codes

LC-3 has three condition code registers:

- **N -- negative**
- **Z -- zero**
- **P -- positive (greater than zero)**

Set by any instruction that writes a value to a register (ADD, AND, NOT, LD, LDR, LDI, LEA)

Exactly one will be set at all times

- **Based on the last instruction that changed a register**

BR: Conditional Branch

Assembler Instruction

BRx LABEL (x CAN BE n, z, p, nz, np, zp, or nzp)

Branch to LABEL iff the selected condition code(s) is(are) set

Encoding

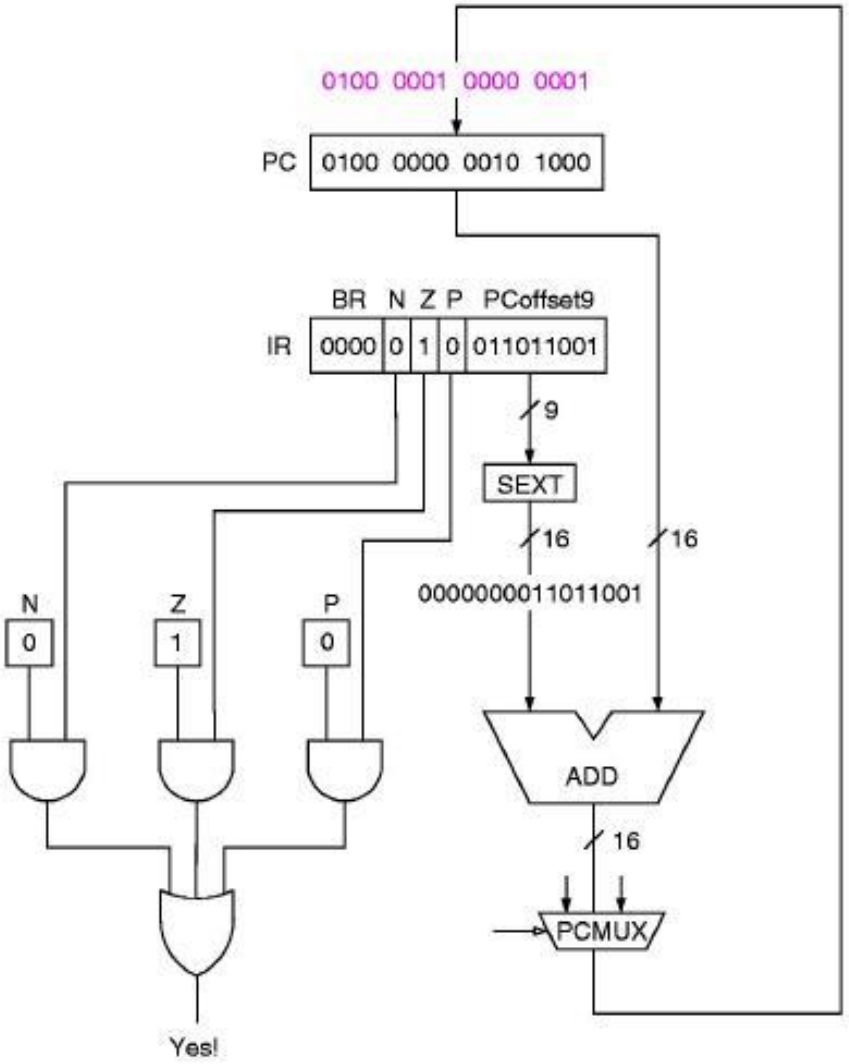
0000 n z p PCoffset9

Example

```
BRzp LOOP   ;branch to LOOP if previous op returned zero or positive
```

BR Data Path

BRz x0D9 → 0000 0 1 0 011011001



Using Branch Instructions

COMPUTE SUM OF 12 INTEGERS USING A COUNTER

JAVA CODE

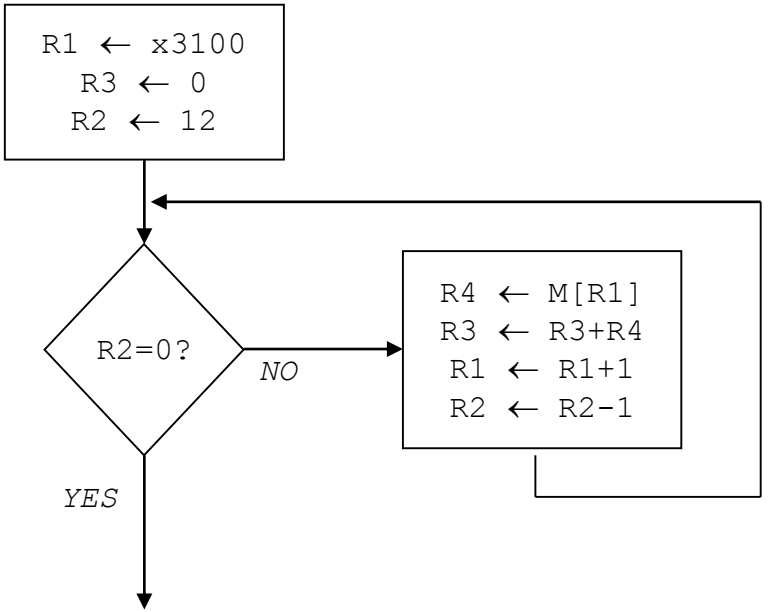
```
int a[12], sum, cnt, i;

i = 0;
sum = 0;

for (cnt=12; cnt!=0; cnt--){
   sum = sum + a[i];
   i = i+1;
}
```

Assume:

- **Numbers start at location x3100.**
- **Instructions start at location x3000.**



Sample Program

Address	*Instruction*	*Assembler Instr.*
x3000	1 1 1 0 0 0 1 0 1 1 1 1 1 1 1 1	*LEA R1, x0FF*
x3001	0 1 0 1 0 1 1 0 1 1 1 0 0 0 0 0	*AND R3, R3, #0*
x3002	0 1 0 1 0 1 0 0 1 0 1 0 0 0 0 0	*AND R2, R2, #0*
x3003	0 0 0 1 0 1 0 0 1 0 1 0 1 1 0 0	*ADD R2, R2, #12*
x3004	0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1	*BRz #5*
x3005	0 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0	*LDR R4, R1, #0*
x3006	0 0 0 1 0 1 1 0 1 1 0 0 0 1 0 0	*ADD R3, R3, R4*
x3007	0 0 0 1 0 0 1 0 0 1 1 0 0 0 0 1	*ADD R1, R1, #1*
x3008	0 0 0 1 0 1 0 0 1 0 1 1 1 1 1 1	*ADD R2, R2, # -1*
x3009	0 0 0 0 1 1 1 1 1 1 1 1 1 1 0 1 0	*BRnzp # -6*

Sample Program

Address	*Instruction*	*Comments*
x3000	1 1 1 0 0 0 1 0 1 1 1 1 1 1 1 1	*R1 ← x3100 (PC+0xFF)*
x3001	0 1 0 1 0 1 1 0 1 1 1 0 0 0 0 0	*R3 ← 0*
x3002	0 1 0 1 0 1 0 0 1 0 1 0 0 0 0 0	*R2 ← 0*
x3003	0 0 0 1 0 1 0 0 1 0 1 0 1 1 0 0	*R2 ← 12*
x3004	0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1	*If Z, goto x300A (PC+5)*
x3005	0 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0	*R4 ← M[R1]*
x3006	0 0 0 1 0 1 1 0 1 1 0 0 0 1 0 0	*R3 ← R3+R4*
x3007	0 0 0 1 0 0 1 0 0 1 1 0 0 0 0 1	*Increment R1 (pointer)*
x3008	0 0 0 1 0 1 0 0 1 0 1 1 1 1 1 1	*Decrement R2 (counter)*
x3009	0 0 0 0 1 1 1 1 1 1 1 1 1 1 0 1 0	*Goto x3004 (PC-6)*

Using Branch Instructions

COMPUTE SUM OF INTEGERS USING A SENTINEL

JAVA CODE

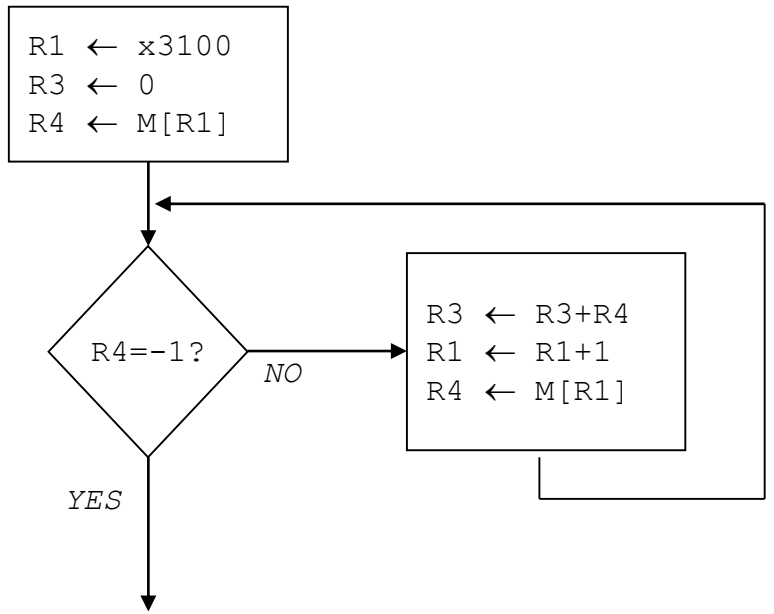
```
int a[100], sum, i;

i = 0;
sum = 0;
value = a[i];

while (value != -1){
    sum = sum + value;
    i = i+1;
    value = a[i];
}
```

Assume:

- **Numbers start at location x3100.**
- **Instructions start at location x3000.**



Sample Program

Address	*Instruction*	*Assembler Instr.*
x3000	1 1 1 0 0 0 1 0 1 1 1 1 1 1 1 1	*LEA R1, x0FF*
x3001	0 1 0 1 0 1 1 0 1 1 1 0 0 0 0 0	*AND R3, R3, #0*
x3002	0 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0	*LDR R4, R1, #0*
x3003	0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0	*BRn #4*
x3004	0 0 0 1 0 1 1 0 1 1 0 0 0 1 0 0	*ADD R3, R3, R4*
x3005	0 0 0 1 0 0 1 0 0 1 1 0 0 0 0 1	*ADD R1, R1, #1*
x3006	0 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0	*LDR R4, R1, #0*
x3007	0 0 0 0 1 1 1 1 1 1 1 1 1 1 0 1 1	*BRnzp #-5*

Sample Program

Address	*Instruction*	*Comments*
x3000	1 1 1 0 0 0 1 0 1 1 1 1 1 1 1 1	*R1 ← x3100 (PC+0xFF)*
x3001	0 1 0 1 0 1 1 0 1 1 1 0 0 0 0 0	*R3 ← 0*
x3002	0 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0	*R4 ← M[R1]*
x3003	0 0 0 0 1 0 0 0 0 0 0 0 1 0 0	*If N, goto x3008 (PC+4)*
x3004	0 0 0 1 0 1 1 0 1 1 0 0 0 1 0 0	*R3 ← R3+R4*
x3005	0 0 0 1 0 0 1 0 0 1 1 0 0 0 0 1	*R1 ← R1+1*
x3006	0 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0	*R4 ← M[R1]*
x3007	0 0 0 0 1 1 1 1 1 1 1 1 1 1 0 1 1	*Goto x3003*

JMP: Jump or Go To

Assembler Instruction

JMP BaseR

Take the next instruction from the address stored in BaseR

Encoding

1100 000 BaseR 00 0000

Example

```
JMP R5   ; if R5 = x3500, the address x3500 is written to the PC
```